— Assignment 1 —
Happy, Sad, Surprised, or Mad?

Tommy Löfstedt, Polina Kurtser, and Brendan Michael

Deadline on April 10, 2026

How good are neural networks at reading faces? You may be able to provide an answer after this assignment. We will consider four basic emotions—*happy*, *sad*, *surprised*, and *mad*—and your task is to train a neural network model to classify images into one of those four emotions.

We provide a dataset with a total of 10,000 images of human faces, with 2,500 images for each of the four emotions. The data are provided on the CS computer systems in the folder (split into train, validation, and test data):

`/import/course/5dv236/vt26/AffectNet/`

The data are a downscaled subset of the AffectNet dataset, about which you find more information here. See Figure 1 for some example images.

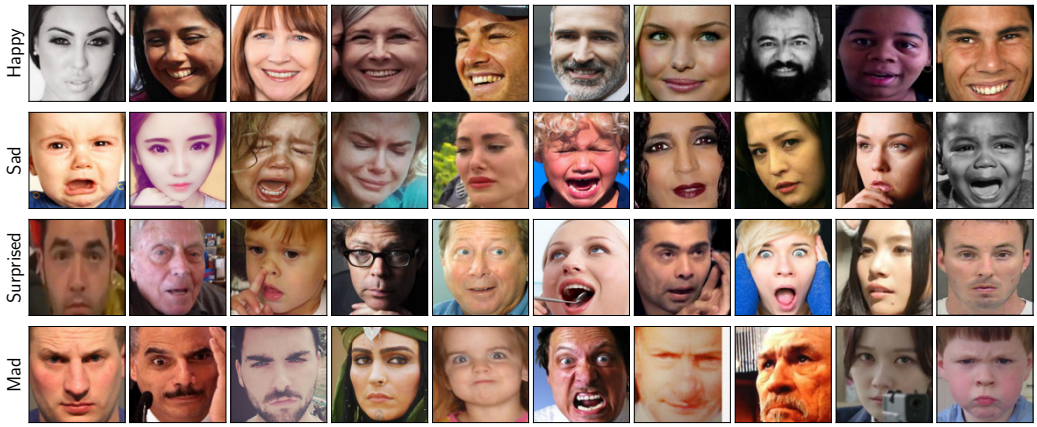

Figure 1: Example faces from the AffectNet dataset. The four rows each correspond to one of the four emotions that we will consider.

Note: We are allowed to use these data under an agreement with the authors, and the agreement does not allow us to share the data outside of the teaching situation here. Therefore, you are **not** allowed to download these data and put them anywhere else. They must stay on university computers.

The solution should work with Keras (with the Tensorflow backend). See separate instructions for how to enable the provided conda environment here. You may export and install the environment on your own computer. You may develop your solution using whatever deep learning library you want, but the version you hand in must run with Keras/Tensorflow using the provided environment/versions to pass the assignment.

You may implement any neural network architecture you want, but you will likely perform well if you implement some form of *convolutional neural network*. (Feel free to try out other types of networks and play around with architectures if you want to experiment!) The network should perform *multi-class classification* into one of four classes (happy, sad, surprised, or mad). In addition to the loss function that you choose, make sure to add *accuracy* as a metric, for the model to report. You may use the provided data loader in the template file `Lab_1-Skeleton_Template.py`. Are you able to reach a *validation* accuracy above 0.7?

You are only allowed to implement the required neural network yourselves. For instance, it is not allowed to use someone else's implemented architecture and it is not allowed to use pre-trained networks from Keras or other deep learning libraries. Please also take note of the course's "AI" policy, which is stated on the course web page[1].

We will evaluate your models on separate test data. There is no strict threshold accuracy you need to reach, but your model must perform clearly above chance levels to pass the assignment (the baseline/chance level accuracy is 0.25, and you need to be clearly above this level).

What to hand in:

- Your trained model (see below how). It *must* load in the conda environment using `tensorflow.keras.models.load_model(...)`. (Make sure to set the file rights so that we can read it (copy it using `cp`). You could use either `chmod` or `setfacl` for this.)

- A short report in PDF format (*follow the template given below*) that

[1] https://www.canvas.umu.se/courses/20985

contains:

- A *short* description of your solution (about 1/3 to 1/2 page).
- The *validation* and final *test* accuracies for *each* of the four classes.
- A figure with the learning curves for your *training and validation* loss and for your *training and validation* accuracies.
- A figure that illustrates your architecture.
- The path to your Keras model on the CS computer systems.

Don't hesitate to contact us, either through email, the course web discussion board, or at the lectures, if you have any questions!

Good luck, and have fun!

Assignment 1: Report Template

Student Name: _____
CS username: _____

Solution Description

Give a short description (about 1/3 or 1/2 page) of your solution. Try to answer the following questions in your description:

- What type of neural network architecture did you choose? Why did you choose it?
- How many and what type of layers did you include? Why did you choose these specific layers?
- What activation functions did you use for your network? Why?
- Did you use any regularization techniques (*e.g.*, Dropout, Batch normalization, ℓ_2 norm, *etc.*)? Why/why not?
- What preprocessing steps, if any, did you apply to the data before feeding them into the model?
- What were the hyperparameters of your model? How did you choose their values?

Validation and Test Accuracies

- Happy: _____ (Validation), _____ (Test)
- Sad: _____ (Validation), _____ (Test)
- Surprised: _____ (Validation), _____ (Test)
- Mad: _____ (Validation), _____ (Test)

Learning Curves and Model Architecture

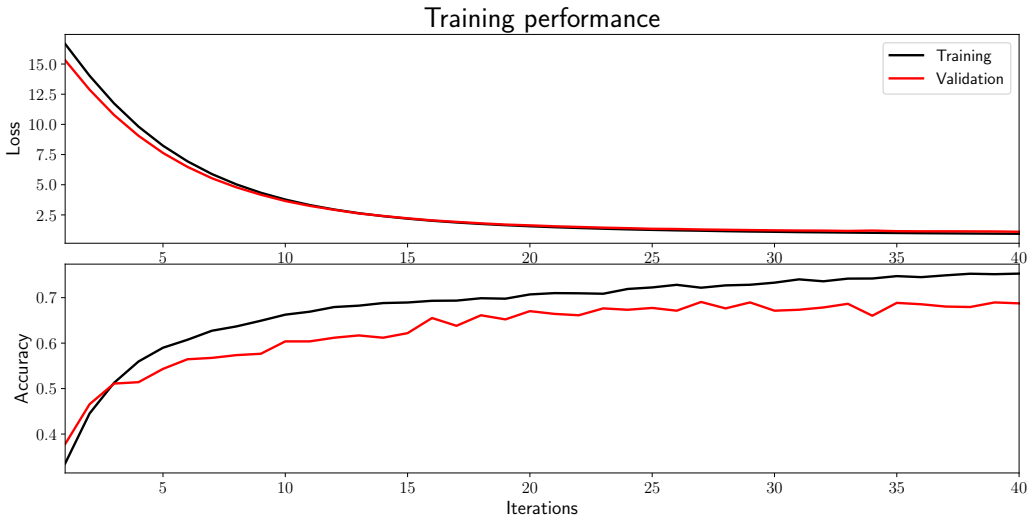

Figure 2: Sample learning curves for training and validation loss and accuracy

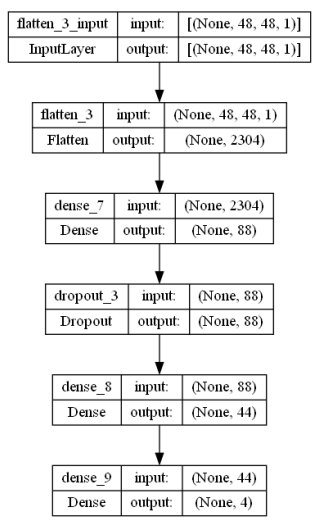

Figure 3: Model architecture (just an example!)

Model Path

Path to the model on the CS computer systems: `/path/to/your/keras/model.h5`.